

Exploring Hallucination Detection in Large Language Models

Team - 111

Ajay Rajendra Kumar | Vicky Daniel | Vaibhav Sankaran

CS6120 Natural Language Processing

Problem Statement and Motivation

- **What is hallucination in LLMs?**
 - LLMs generate fluent but factually incorrect or unsupported text
 - Particularly harmful in open-domain QA where accuracy is critical
- **Why is it hard to detect?**
 - Existing methods rely on repeated sampling which is expensive and slow
 - Post-processing approaches add latency at inference time
 - Supervised methods require costly human-annotated hallucination labels
- **Our motivation**
 - Detect hallucinations in real-time during inference with no annotation cost
 - LLMs may already encode uncertainty in their internal representations
 - Build a lightweight, model-agnostic detection and mitigation framework

Prior Work Review

- **Sampling-based methods (SelfCheckGPT)**
 - Generate multiple outputs and compare consistency across samples
 - **High compute cost** - requires N forward passes per query
- **Retrieval-Augmented Generation (RAG)**
 - Grounds answers in external documents to reduce fabrication
 - Prevents hallucination but does not detect it post-generation
- **Probing internal states (ITI - Inference-Time Intervention)**
 - Uses hidden states to predict truthfulness
 - Supervised - depends on labeled training data per model
- How about we use internal states of LLMs to guide hallucination detection?

Base Paper: The MIND Framework

Unsupervised Real-Time Hallucination Detection based on the Internal States of Large Language Models

Weihsang Su^{*1}, Changyue Wang^{†1}, Qingyao Ai^{‡1}, Yiran Hu¹, Zhijing Wu², Yujia Zhou³
Yiqun Liu¹

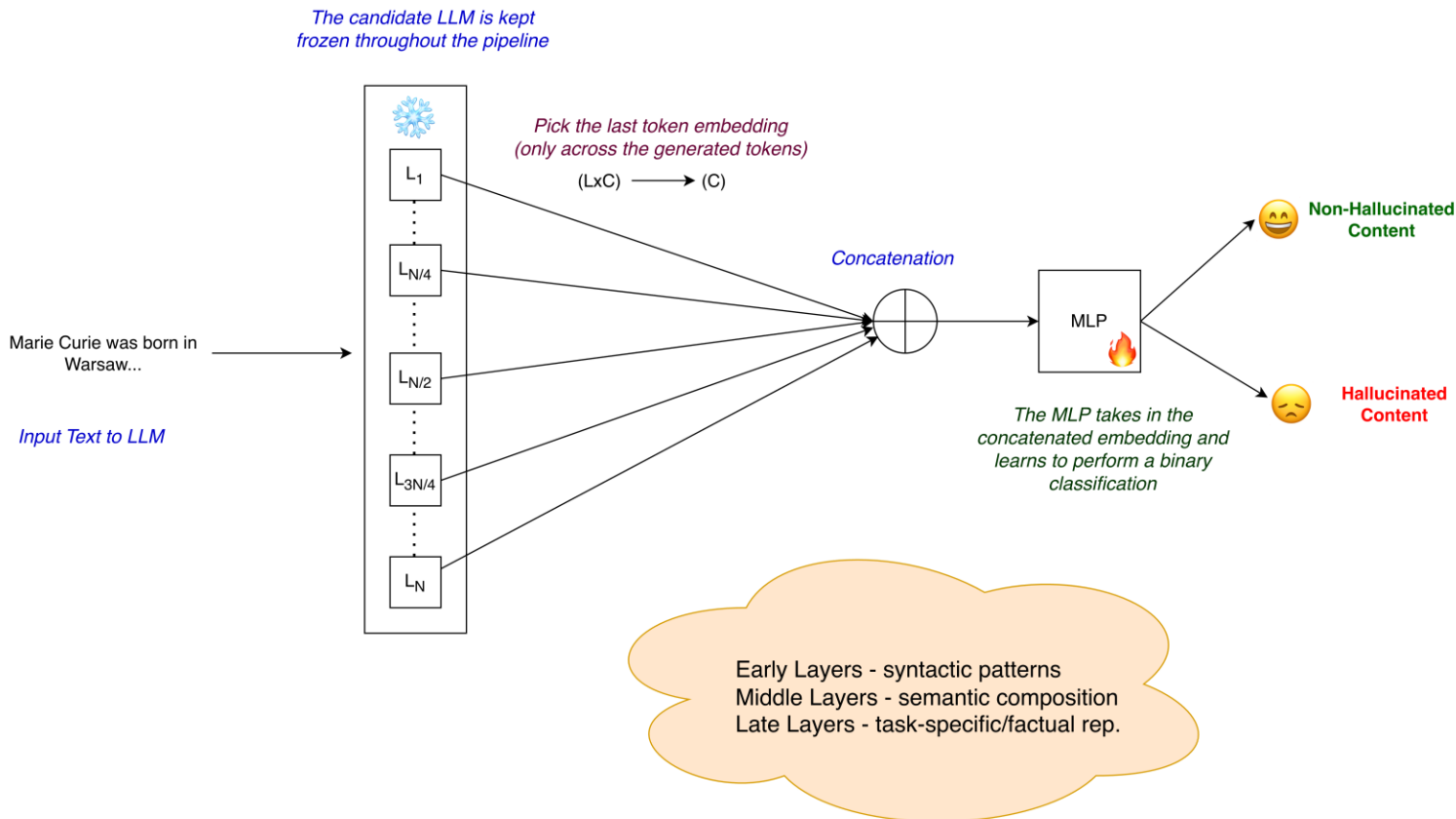
¹Department of Computer Science and Technology, Tsinghua University

²School of Computer Science and Technology, Beijing Institute of Technology

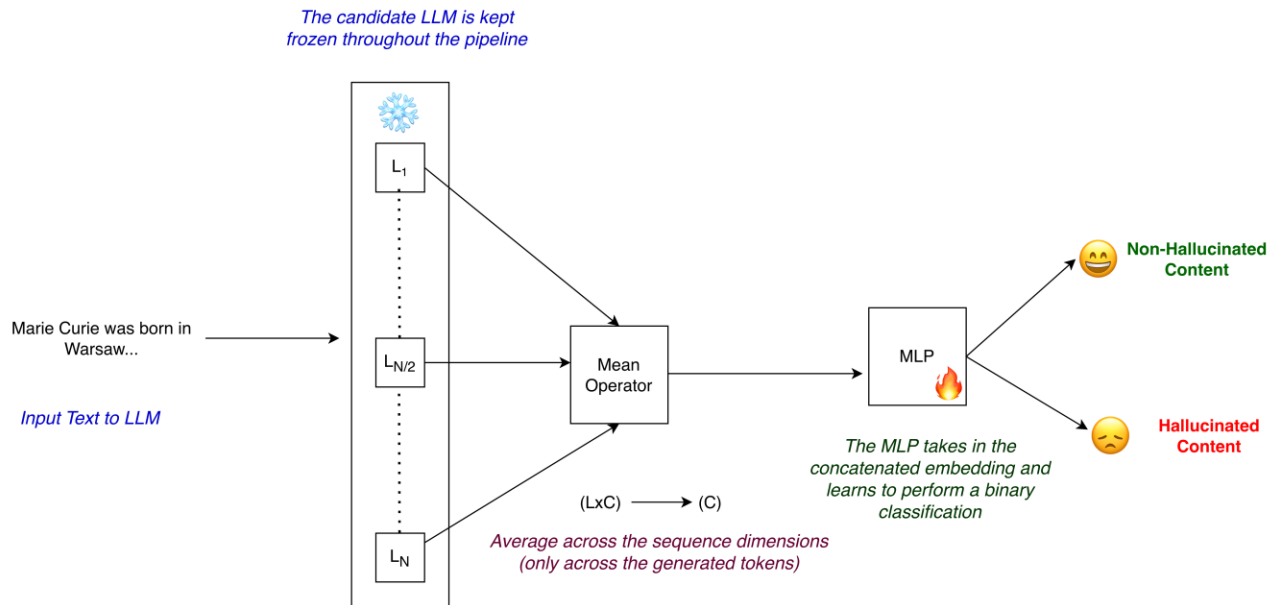
³School of Information, Renmin University of China

- Uses internal states of the model to detect hallucinations
- Key insight: Aggregate last token embedding of every layer
- Self-supervised data generation
- Train a light-weight MLP to perform classification
- Predicts hallucination at sentence level during generation

Method-1

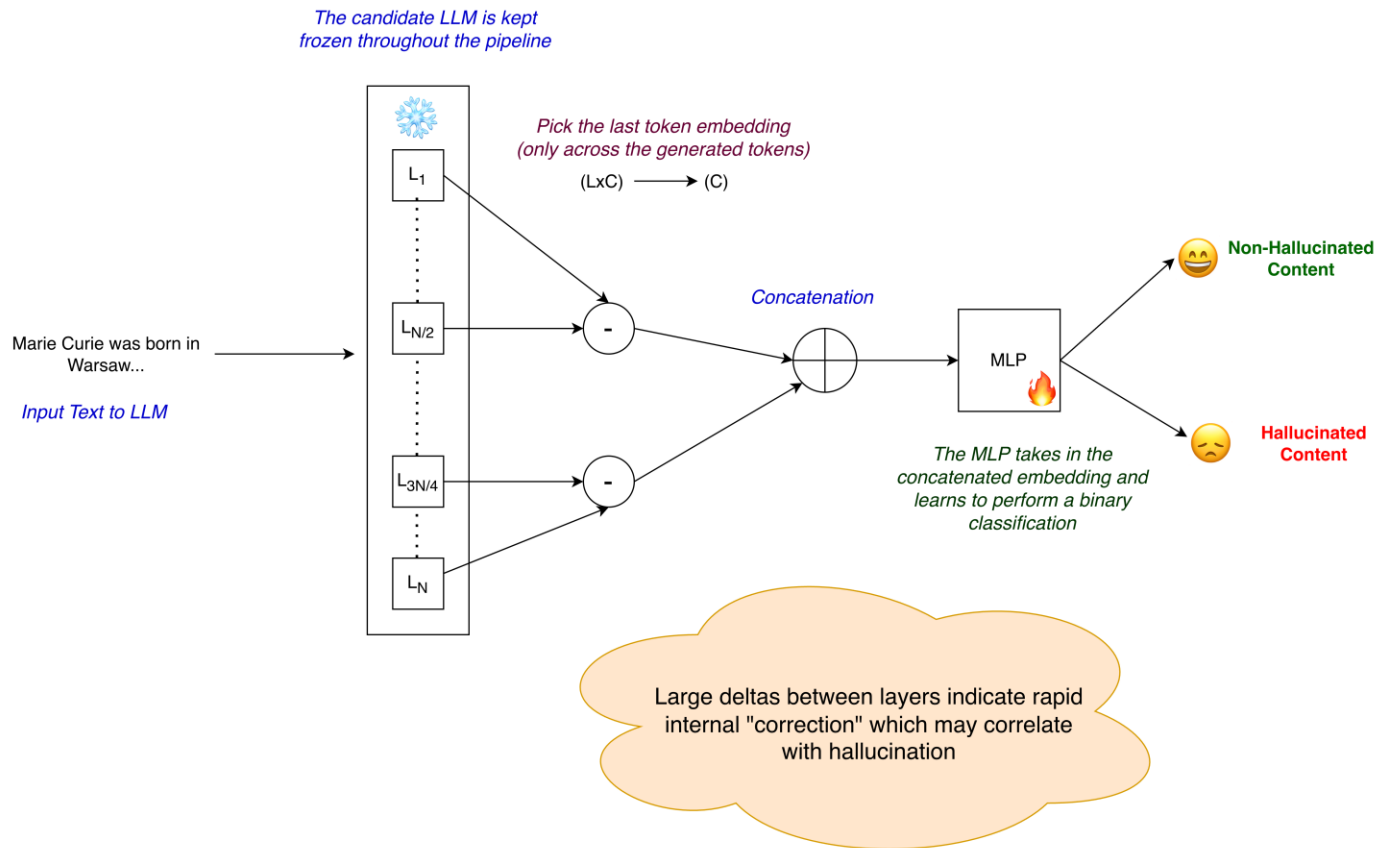


Method-2

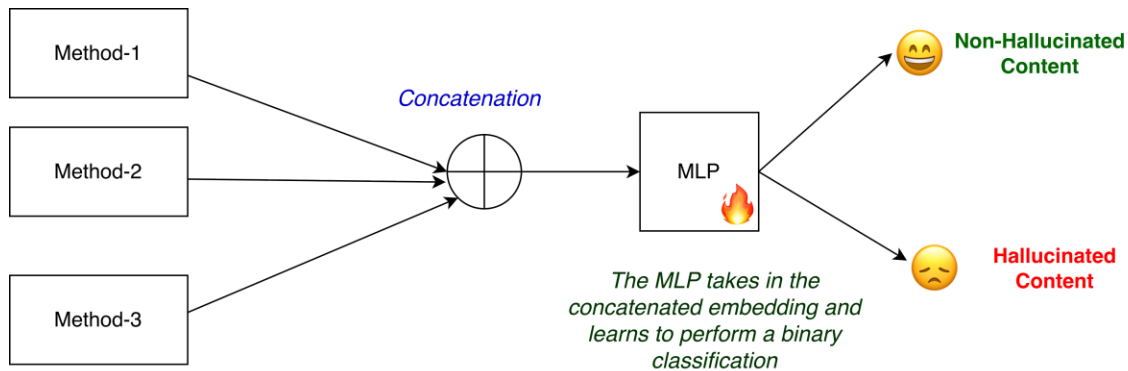


Select a sparse set of layers and compute the average of their token embeddings across the answer tokens

Method-3

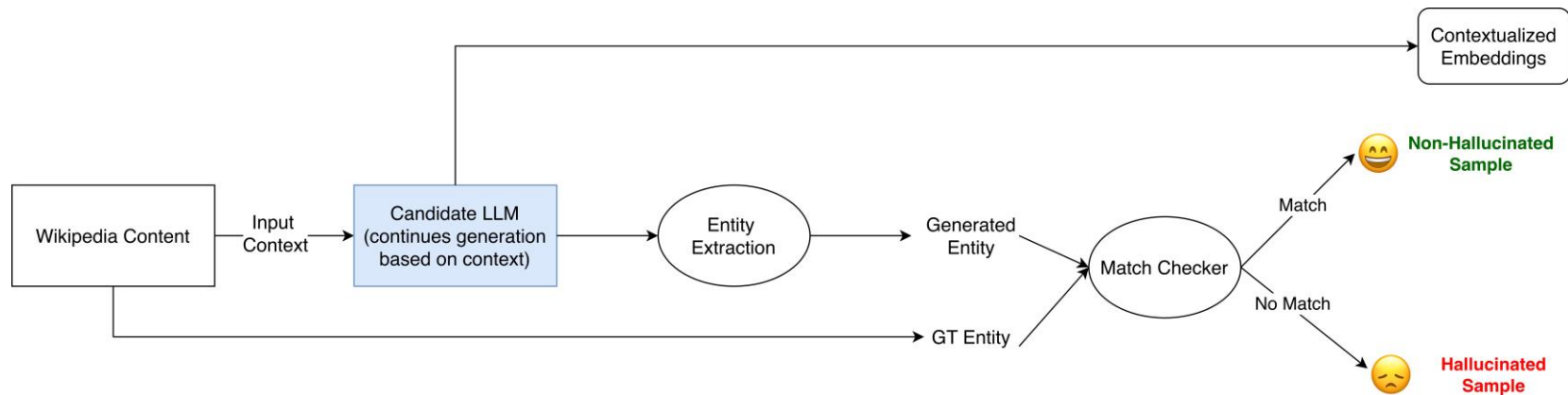


Method-4



How about we give all the signals to the MLP and let it decide which one it finds more useful?

Training Data Methodology: Self-Supervised Pipeline

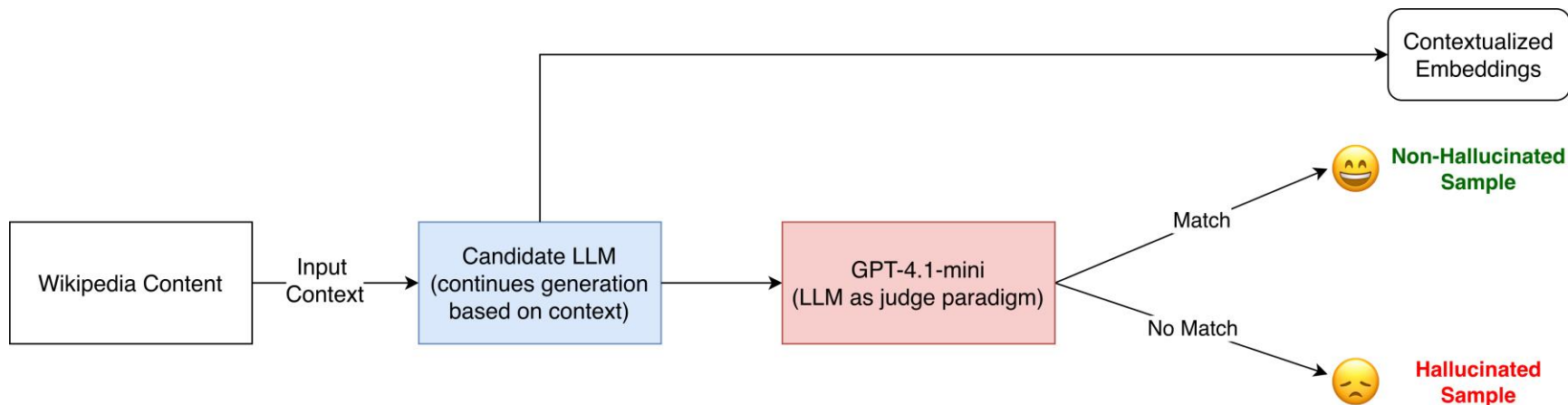


- **Framework:** Adapted from the MIND architecture
- **Dynamic Generation:** Creates model-dependent training data on the fly
- **Pipeline Components:**
 - Extraction: Powered by spaCy for robust entity recognition
 - Validation: Match-checker verifies generated entities against Ground Truth (GT)

Table 2: Dataset split counts for each model.

Model	Split	Right	Hallu	Total
llama3base8b	train	6000	2061	8061
llama3base8b	valid	1000	362	1362
llama3base8b	test	1304	430	1734
gptj7b	train	6000	3709	9709
gptj7b	valid	1000	657	1657
gptj7b	test	1304	795	2099

HELM Benchmark Generation



- **Input Source:** Uses Wikipedia Content ([WikiText-103](#)) to establish baseline context
- **Candidate Generation:** The target LLM continues generation while extracting contextualized embeddings
- **Evaluation:** Employs GPT-4.1-mini under an LLM-as-judge paradigm
- **Sample Classification:**
 - Match: Categorized as a Non-Hallucinated Sample
 - No Match: Categorized as a Hallucinated Sample

Results & Findings: Model-Dependent Performance

Strategy	gptj7b	llama3base8b	Strategy	gptj7b	llama3base8b
MIND (original)	88.17	<u>53.52</u>	MIND (original)	<u>96.69</u>	90.81
multi_layer_mean	72.60	54.39	multi_layer_mean	97.93	93.17
multi_layer	69.87	52.32	multi_layer	94.93	<u>92.08</u>
multi_layer_last_token	68.84	50.49	multi_layer_deltas	94.31	90.53
multi_layer_deltas	68.02	51.18	multi_layer_last_token	94.05	90.53
HELM Sentence Level AUC (↑)			HELM Passage Level AUC (↑)		

Strategy	gptj7b	llama3base8b	Strategy	gptj7b	llama3base8b
MIND (original)	0.46	<u>0.16</u>	MIND (original)	0.51	<u>0.29</u>
multi_layer_mean	0.23	0.20	multi_layer_mean	0.31	0.29
multi_layer	0.16	0.14	multi_layer	0.11	0.21
multi_layer_deltas	0.15	0.11	multi_layer_deltas	0.06	0.13
multi_layer_last_token	0.13	0.11	multi_layer_last_token	0.03	0.10
HELM Sentence Level Corr (↑)			HELM Passage Level Corr (↑)		

Table 1: Across all four tables, the best-performing values are bolded and the second-best values are underlined within each model column.

Evaluation Metrics

- AUC (Area Under Curve) - Threshold-independent measure of classification performance
- Correlation - Measures alignment between classifier confidence and ground truth

Key Findings

- **Model-dependent behavior is dominant:** No single strategy wins across the board
- **MIND is strong but fragile:** Dominates on GPT-J but fails to generalize to Llama-3
- **multi_layer_mean is the most robust:** The only method that consistently performs best on Llama-3 across all metrics

Results & Findings: Analyzing Feature Strategies

Strategy	gptj7b	llama3base8b	Strategy	gptj7b	llama3base8b
MIND (original)	88.17	<u>53.52</u>	MIND (original)	<u>96.69</u>	90.81
multi_layer_mean	<u>72.60</u>	54.39	multi_layer_mean	97.93	93.17
multi_layer	69.87	52.32	multi_layer	94.93	<u>92.08</u>
multi_layer_last_token	68.84	50.49	multi_layer_deltas	94.31	90.53
multi_layer_deltas	68.02	51.18	multi_layer_last_token	94.05	90.53
HELM Sentence Level AUC (↑)			HELM Passage Level AUC (↑)		
Strategy	gptj7b	llama3base8b	Strategy	gptj7b	llama3base8b
MIND (original)	0.46	<u>0.16</u>	MIND (original)	0.51	<u>0.29</u>
multi_layer_mean	0.23	0.20	multi_layer_mean	0.31	0.29
multi_layer	0.16	0.14	multi_layer	0.11	0.21
multi_layer_deltas	0.15	0.11	multi_layer_deltas	0.06	0.13
multi_layer_last_token	0.13	0.11	multi_layer_last_token	0.03	0.10
HELM Sentence Level Corr (↑)			HELM Passage Level Corr (↑)		

Table 1: Across all four tables, the best-performing values are bolded and the second-best values are underlined within each model column.

- **Aggregation function is critical**
 - o Mean pooling (multi_layer_mean) significantly outperforms naive concatenation (multi_layer)
 - o Takeaway: How representations are combined matters much more than simply increasing the number of representations used
- **Consistently weak strategies**
 - o Delta-based features (multi_layer_deltas)
 - o Last-token representations (multi_layer_last_token)

Discussion: Challenges Faced

- **Primary Challenges**
 - **Data Scarcity:** Lack of existing HELM benchmark data for our target models
 - **Resource Constraints:** Compute bottlenecks when running large-scale models, coupled with budget limitations for OpenAI API annotations
 - **Engineering Overhead:** Extensive codebase refactoring required to integrate custom pipeline implementations
- **Future Scope**
 - **Architectural Expansion:** Experiment with more complex classifier designs
 - **Model Diversity:** Broaden evaluation across a wider range of foundation models

Individual Contributions

Ajay Rajendra Kumar

- Method 1
- Method 4

Vicky Daniel

- Method 2
- Self-Supervised Data Generation Pipeline

Vaibhav Sankaran

- Method 3
- HELM Benchmark Generation

- Literature review and prior work analysis
- Experiment design and evaluation methodology
- Results analysis and interpretation
- Report and Presentation preparation
- Codebase development and debugging

Thank You!